Computational Physics 2

Lecturer: me

Outline of module

- Random numbers and stochastic processes
- Monte Carlo methods
- Linear algebra
- Minimisation
- Partial differential equations (PDEs)

Assessment: Continuous assessment (3 quizes) 30%
Exam 70%

MP468P Project: Oct–May

Overview

1. Random numbers
   - Random number generators
   - Statistical distributions
   - Transformation method
   - Rejection method

2. Summary

Random numbers

Why should we want random numbers?

- Simulate stochastic processes in nature
  - Brownian motion, crystal growth
  - Statistical physics, thermodynamics
  - Quantum mechanics, quantum field theory
  - Evolutionary processes, population dynamics
  - Stock markets, financial markets
- To simulate 'unknowns'
- Randomised control trials, statistical analysis
- Monte Carlo integration
- Search algorithms

Random number generators

Some numbers are more random than others

No classical computer can create truly random numbers
— only physical processes can do that

- Throw dice, flip coins, roll roulette wheels
- Get 'noise' from the environment
- Or use series of numbers that only 'look random'

Pseudo-random number generators

All prngs use (integer) arithmetic to produce series of numbers
The series must repeat itself after a finite number of steps

In computational physics we may need vast numbers of random numbers

- The period must be as long as we can get it
- There must be no 'hidden' correlations among numbers

Caveat emptor

Linear congruential algorithm

Simple, traditional algorithm: $X_{n+1} = (aX_n + c) \mod m$
a, c and m are integers. The period is at most m.

Correlations: Group into vectors: $\vec{x}_n = (x_n, x_{n+1}, x_{n+2})$.
Then the $\{\vec{x}_n\}$ will lie in distinct planes. (Marsaglia, 1968)

Considered unsuitable for serious Monte Carlo work

Modern algorithms

Mersenne twister, xorshift, ...
Even the best don't always pass all randomness tests
Good news: `numpy` uses good pnrg, based on Mersenne twister algorithm.

Lesson

Be suspicious of random number generators.
Make sure the one you use is good enough for your purpose.

PRNG's in python

Two different options :-(

package random

random.random() — return uniform real number in $[0.0, 1.0)$

random.gauss(mu, sigma) — return normally distributed real number with mean mu and width (standard dev) sigma.

random.randint(a,b) — return random integer $N \in [a, b]$

package numpy.random

numpy.random.rand() — uniform real number in $[0.0, 1.0)$

numpy.random.randn() — normally distributed real number, mean 0.0, st.dev. 1.0.

PRNG's in python

Two different package options :-(

random or numpy.random

Suggestion

Pick one and use it —
don't use both packages in the same code unless you really have to.
Maybe numpy.random has more options

Why? Scientific programming was not the top priority for python language. (Contrast: Fortran, matlab, julia languages)

Seeding

The 'seed' gives the starting point for the series

- If you want two identical sets of 'random' numbers, start with the same seed (eg to check your code)
- if you want two different sets of pseudo-random numbers, make sure you start with different seeds.

Python

Use	`random.seed(x)`	to set the state (seed)
	`random.seed()`	to set a seed based on the current time (useful for producing different numbers each
	`random.getstate()`	to save the current state of the rng
	`random.setstate()`	to reset to a saved state

Statistical distributions

Most rngs produce a uniform distribution between 0 and 1
[or integers between 0 and `RAND_MAX`]

$$P(X \in [x_1, x_1+\Delta x]) = P(X \in [x_2, x_2+\Delta x]) = \Delta x \quad \forall (x_1, x_2) \in \langle 0, 1-\Delta x \rangle$$

We may want different distributions:

- exponential
- gaussian
- poisson
- linear
- more complicated, in one or more dimensions

Normalisation

All distributions must obey $\int_{-\infty}^{\infty} P(x)dx = 1$

Two general methods

- transformation
- rejection

Transformation method

Suppose X is uniformly distributed, take $Y = f(X)$.

How is Y distributed?

The probability of finding X in $(x, x + dx)$ must be the same as the probability of finding Y in $(y, y + dy)$,

$$|P_Y(y)dy| = |P_x(x)dx|$$

This gives us

$$P_Y(y)|f'(x)dx| = P_x(x)|dx| \quad \Longrightarrow \quad P_Y(y) = \frac{P_x(x)}{|f'(x)|}$$

Stochastic variables

Note the difference between X and x:

X is a stochastic variable — takes random values

x is an ordinary variable — the argument of the probability distribution

Transformation method

Example

$$P_X(x) = \begin{cases} 1 & 0 < x < 1 \\ 0 & \text{otherwise} \end{cases}$$

$$Y = f(X) = -\ln X \quad \Longrightarrow \quad 0 < Y < \infty$$

$$P_Y(y) = \frac{1}{|f'(x)|} = x = e^{-y}$$

Y is exponentially distributed

Example

$$Y = \sqrt{X} \quad \Longrightarrow \quad P_Y(y) = \frac{1}{1/2\sqrt{x}} = 2\sqrt{x} = 2y$$

You can check that both these are correctly normalised.

Obtaining a specific distribution

We want a certain $p(y)$. What is $y = f(x)$ if x is uniform?

$$\frac{1}{f'(x)} = \frac{dx}{dy} = p(y) \quad \Longrightarrow \quad dx = p(y)dy$$

$$\Longrightarrow \quad x = \int_{-\infty}^{y} p(z)dz \equiv \mathcal{C}(y)$$

Inverting this gives us: $y(x) = \mathcal{C}^{-1}(x)$

x is uniformly distributed.

What transformation $y = f(x)$ will provide variable y with distribution $p(y)$?

$$\mathcal{C}(y) = \int_{-\infty}^{y} p(z)dz \qquad y = f(x) = \mathcal{C}^{-1}(x)$$

Obtaining a specific distribution

x is uniformly distributed.

What transformation $y = f(x)$ will provide variable y with distribution $p(y)$?

$$\mathcal{C}(y) = \int_{-\infty}^{y} p(z)dz \qquad\qquad y = f(x) = \mathcal{C}^{-1}(x)$$

We can find the transformation function if

1. we can integrate our distribution $\rightarrow$ cumulative distribution $\mathcal{P}(y)$
2. we can invert the cumulative distribution function

analytically

Very useful for scaling and shifting distributions

Shifting and scaling

The transformation method is useful for shifting and scaling distributions:

$$y = x/a + b \quad \Longrightarrow \quad P_y(y) = aP_x(x) = aP_x\big(a(y-b)\big)$$

Example

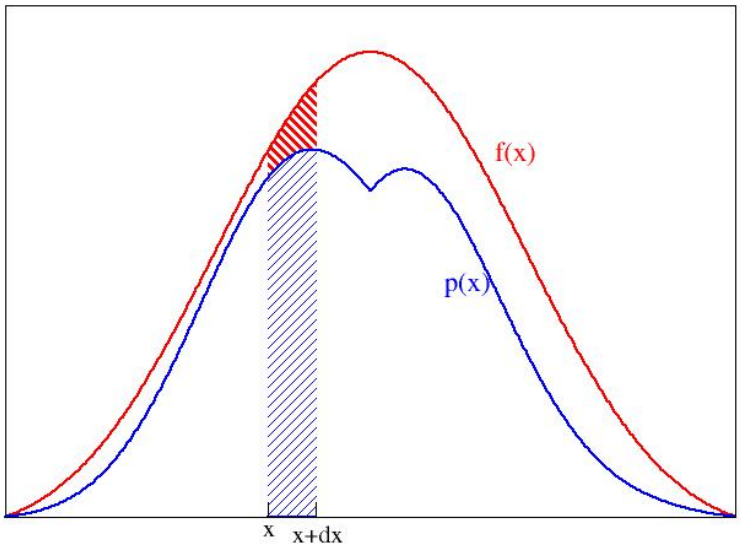
X is gaussian with average 0 and variance 1.
We want Y to be gaussian with average μ and variance σ^2,

$$P_Y(y) = \frac{1}{\sigma\sqrt{2\pi}} e^{-(y-\mu)^2/2\sigma^2}$$

We achieve this by $Y = \sigma X + \mu$

Rejection method

What if we cannot integrate or invert?



The shaded blue area is the prob of finding X between x and $x + dx$

Can we find a way of picking (X, Y) uniformly distributed under the blue curve?

Assume we can do this for $f(x)$.

The ratio of areas is $p(x)/f(x)$

Rejection method

Implementation

1. Pick uniform $Z \in \langle 0, A \rangle$; $A = \int_{-\infty}^{\infty} f(x)dx$
2. Find $X = F^{-1}(Z)$ where $F(y) = \int_{-\infty}^{y} f(x)dx$
3. Pick random uniform $Y \in \langle 0, 1 \rangle$
4. If $Y < p(X)/f(X)$ then accept X as your random number
   else reject X and try again

Summary

- Random numbers are widely used in computational physics
- Good pseudo-random number generators exist, but check before using an inbuilt generator for serious business!
- Transformation method:
  - Obtain new distribution from old analytically
  - Only works for functions where the integral can be obtained and inverted analytically
- Rejection method
  - Can be used for any distribution
  - Pick random numbers distributed under curve $f(x) \geq p(x)$
  - Accept numbers with probability $p(x)/f(x)$.
  - Similar to Monte Carlo integration (next)